1. ABSTRACT

Reporting everyday civic problems, such as potholes, broken streetlights, water pipeline leaks, overflowing garbage bins, and sparking electrical wires, is often slow and frustrating. Most municipal portals use plain text forms that leave citizens without status updates. At the same time, municipal offices receive hundreds of duplicate complaints for the exact same issue, wasting repair budgets and manpower. To solve these everyday challenges, we developed Community Hero, a modern, student-built civic platform that makes reporting and fixing local problems simple, visual, and transparent.

The frontend is built using React 18 and Three.js. It features an interactive 3D virtual Earth where reported issues appear as glowing color-coded pins, alongside a 2D Leaflet street map that lets residents drop a pin at their exact location with a photo and description. The backend uses Node.js and Express connected to a lightweight SQLite database.

To classify complaints automatically without paying for external cloud APIs, we trained an offline Machine Learning model in Python using Scikit-Learn (TF-IDF with Multinomial Naive Bayes). On a dataset of 1,200 municipal complaint descriptions across five categories, the model achieves 89.2% accuracy in under 5 milliseconds. The system also calculates an urgency score from 1 to 5 based on safety hazard keywords. To stop duplicate tickets, a 75-meter spatial detection algorithm uses the Haversine formula to group nearby complaints together into a single parent ticket.

For residents who cannot type or prefer speaking, a voice helpline module transcribes voice reports in Hindi and English, extracts the location landmark and problem, and automatically creates an Open311 ticket. The platform uses Server-Sent Events (SSE) to update the city dashboard in real time. Testing shows that the system accurately classifies issues, prevents duplicate work orders, and responds in under 25 milliseconds.

Keywords: Civic-Tech, 3D Globe, Three.js, React, Node.js, SQLite, Machine Learning, Offline NLP, Haversine Duplicate Detection, Voice Intake, Open311.

2. INTRODUCTION

2.1 Background and Real-World Context

In Indian cities, everyday infrastructure issues such as broken roads, malfunctioning streetlights, leaking water pipelines, open drains, and garbage dumps create significant safety hazards and citizen frustration. Traditional grievance methods, including physical visits, busy helplines, and unstructured web portals, are slow and rarely provide transparent status updates. Concurrently, municipal authorities receive hundreds of redundant complaints during monsoon or severe weather for identical incidents, resulting in duplicated inspections, wasted staff hours, and inefficient resource allocation.

2.2 Need for Voice Accessibility

Many elderly or non-technical residents struggle with smartphone apps or typing detailed complaints in English. A voice option allows anyone to simply speak their problem in Hindi or English over the phone or through their browser microphone. The system extracts the location landmark and the problem automatically, making civic reporting accessible to everyone.

2.3 Project Motivation and Objectives

As undergraduate computer science students, our goal was to build a modern, visual, and practical smart city civic reporting platform. The core objectives include: (1) Replacing flat tabular forms with an interactive 3D Virtual Earth digital twin and 2D street map; (2) Training an offline Machine Learning model to categorize complaints and rate urgency without expensive cloud APIs; (3) Implementing a 75-meter spatial duplicate detection algorithm using the Haversine formula; and (4) Providing a multilingual voice helpline for accessibility in Hindi and English.

2.4 Scope of Deliverables

Phase III deliverables encompass: a responsive React 18 frontend with Three.js WebGL and Leaflet mapping, an Express.js backend with SQLite WAL storage, a trained Scikit-Learn NLP text classification pipeline, 75-meter spatial deduplication, a multilingual voice intake module, Server-Sent Events (SSE) for live telemetry, and community gamification.

3. LITERATURE REVIEW

3.1 Existing Civic Complaint Systems

Platforms like FixMyStreet in the United Kingdom and SeeClickFix in North America demonstrated that allowing citizens to report local issues directly on maps speeds up municipal repairs [1], [3]. These initiatives established open frameworks that helped shape the Open311 GeoReport v2 standard [2]. However, these systems rely primarily on 2D form inputs, do not offer interactive 3D visualizations, lack voice reporting in regional languages, and do not include automated spatial duplicate grouping.

3.2 Machine Learning in Text Classification

Text classification is an established area in Natural Language Processing. Machine learning algorithms such as Multinomial Naive Bayes and Support Vector Machines, when combined with Term Frequency-Inverse Document Frequency (TF-IDF) feature extraction, offer high classification accuracy with low compute requirements [6], [16]. Training our model locally on labeled municipal records avoids external cloud API expenses and provides instant, privacy-preserving classification on the local server [6].

3.3 3D Graphics and Geospatial Digital Twins

Modern web browsers can render hardware-accelerated 3D graphics smoothly using WebGL and Three.js without requiring third-party plugins [4], [9]. Research by Biljecki et al. indicates that 3D city models improve user engagement and cognitive understanding of spatial clusters compared to flat 2D maps [5]. Combining a 3D Earth digital twin with a 2D Leaflet street map [10] provides both a citywide overview and street-level precision.

3.4 Multilingual Voice Telephony in Civic Reporting

Voice intake removes barriers for citizens who find typing in English difficult [3]. Modern web standards such as the W3C Web Speech API [13] and conversational telephony frameworks like Bolna [14] allow spoken words in Hindi and English to be converted into structured data. In combination with Server-Sent Events [7], the system pushes real-time updates directly to administrative dashboards.

3.5 Spatial Duplicate Detection and Clustering

Municipal staff often waste hours investigating duplicate complaints for the exact same physical problem. Geodesic distance calculation via the Haversine formula [15] provides an accurate mathematical model for surface-level proximity verification. By filtering candidates within a 75-meter spatial radius and matching incident categories, the platform clusters related reports under a single master ticket.

3.6 Summary and Research Gap

Table 1 summarizes the existing systems and research gaps addressed by Community Hero [1], [2], [4], [6].

TABLE 1. SUMMARY OF REVIEWED WORKS AND THEIR RELEVANCE TO THIS PROJECT

Platform / Tool	Method Used	Key Strength	Identified Limitation
FixMyStreet / SeeClickFix	2D Map & Web Forms	Pioneered crowdsourced issue reporting	No 3D view, no offline ML, no voice intake
Scikit-Learn NLP	TF-IDF + Naive Bayes	Fast, lightweight offline classification	Needs custom labeled municipal dataset
Three.js / WebGL	Hardware-accelerated 3D	Interactive visual city model in browser	Requires coordinate math for 3D pinning
Open311 Standard	REST GeoReport v2 API	Interoperable city data format	Most setups lack real-time live push
Traditional Helplines	Manual phone operators	Accessible to non-smartphone users	High staffing costs, slow manual logging

4. PROBLEM DEFINITION AND OBJECTIVES

4.1 Problem Statement

Municipal grievance systems are slow, fragmented, and lack real-time transparency. City workers are overwhelmed by duplicate complaints that waste staff time. There is an urgent need for an open, easy-to-use civic platform that automates complaint sorting, blocks duplicate work orders, and provides real-time updates for both citizens and authorities.

4.2 Mathematical Formulation

• **Distance Formula:** 1. Haversine Distance: To find if two reports are within 75 meters on Earth's surface, the system uses the Haversine formula:

$a = \sin^2(\Delta lat / 2) + \cos(lat1) \cdot \cos(lat2) \cdot \sin^2(\Delta lng / 2)$; $c = 2 \cdot \text{atan2}(\sqrt{a}, \sqrt{(1 - a)})$
$d = R \cdot c$ (where R = 6,371,000 meters; threshold $d \leq 75.0$ m)

• **3D Coordinates:** 2. 3D Globe Projection: Latitude (φ) and longitude (λ) in radians are converted to 3D Cartesian coordinates (X, Y, Z) on a sphere of radius R via:

$X = R \cdot \cos(\varphi) \cdot \sin(\lambda)$; $Y = R \cdot \sin(\varphi)$; $Z = R \cdot \cos(\varphi) \cdot \cos(\lambda)$

4.3 System Specifications

Table 2 outlines the system requirements used to build and test Community Hero.

TABLE 2. SYSTEM REQUIREMENTS AND TECHNICAL SPECIFICATIONS

Component	Minimum Requirement	Development Setup
Operating System	Windows 10/11, macOS, Linux	macOS Sonoma / Ubuntu 22.04 LTS
Processor	Dual-core CPU, 2.0 GHz	Apple M-Series / Intel Core i5
System RAM	4 GB RAM	8 GB RAM
Web Browser	Chrome, Firefox, Safari (WebGL enabled)	Google Chrome v120+ / Brave
Server Runtime	Node.js v18+ LTS	Node.js v20+ LTS with Express.js
ML Framework	Python 3.10+ with Scikit-Learn	Python 3.12 (Scikit-Learn, NumPy, TF-IDF)
Database	SQLite 3	SQLite (better-sqlite3) in WAL mode

5. PROPOSED METHODOLOGY AND ARCHITECTURE

5.1 System Architecture Overview

Four-Tier Setup: Community Hero uses a clean, four-tier architecture designed for speed and simplicity:

- **1. Frontend:** Client Tier (Frontend): Single-page React 18 application with Three.js 3D globe, Leaflet 2D street map, and Tailwind CSS.
- **2. Server:** Backend API Tier: Node.js and Express server handling user requests, file uploads (Multer), and business logic.
- **3. Machine Learning:** Machine Learning Tier: Local Python service running our trained Scikit-Learn model to classify text and rate urgency in under 5ms.
- **4. Database:** Persistence Tier: SQLite database in Write-Ahead Logging (WAL) mode for fast, single-file storage.

5.2 3D Globe and 2D Interactive Map

The 3D Globe renders a virtual Earth where issues appear as glowing color-coded pins. Latitude and longitude are converted to 3D sphere coordinates. For precise street-level reporting, users switch to the 2D Leaflet street map to drop a pin at their exact location.

5.3 Trained Machine Learning Model (Offline NLP)

We compiled a custom dataset of 1,200 labeled complaints across five categories: (1) Potholes & Road Damage; (2) Broken Streetlights; (3) Water Leaks & Drainage; (4) Garbage & Sanitation; and (5) Electrical Hazards. The model uses TF-IDF vectorization and a Multinomial Naive Bayes classifier. It runs offline on the local server in under 5 milliseconds. An urgency rating (1 to 5) is calculated using predefined hazard keywords like 'live wire' or 'deep hole'.

5.4 75-Meter Spatial Duplicate Detection

Before inserting a new report into the database, the backend queries active tickets in the same category within a 1-kilometer bounding box. It applies the Haversine formula to compute exact distance. If an active ticket of the same category exists within 75 meters, the system marks the incoming complaint as a duplicate child ticket and links it to the original master ticket. This prevents duplicate dispatch orders while still recording citizen upvotes.

5.5 Database Design and Schema

The SQLite database is structured around five primary tables: (1) users; (2) issues; (3) verifications; (4) audit_trail; and (5) municipal_staff. Table 3 outlines the key schema structures and their integrity rules.

TABLE 3. PRIMARY DATABASE TABLES AND INTEGRITY RULES

Table Name	Purpose	Key Columns	Integrity Rules
users	Stores citizen and staff profiles	id, name, phone, xp_points, badge	id is PRIMARY KEY
issues	Core complaint records with GPS coords	id, title, category, lat, lng, status, urgency, master_id	FOREIGN KEY to users(id)
verifications	Community photo verification votes	id, issue_id, user_id, vote_type, created_at	UNIQUE(issue_id, user_id)
audit_trail	History log of status changes and actions	id, issue_id, action, actor, timestamp	FOREIGN KEY to issues(id)
municipal_staff	Department staff and assignment status	id, name, department, phone, active_tickets	id is PRIMARY KEY

6. PROPOSED ALGORITHMS AND PSEUDOCODE

6.1 Algorithm 1: Trained Model Prediction Algorithm

Algorithm 1 outlines how incoming text complaints are categorized and scored for urgency using our offline trained Scikit-Learn model:

Algorithm 1: Trained Machine Learning Prediction Pipeline

```
ALGORITHM 1: Trained_Model_Prediction(user_text)
INPUT : user_text (String: citizen complaint description)
OUTPUT: predicted_category (String), urgency_score (Integer 1-5)

1. cleaned_text = Preprocess(user_text)  // lowercase, strip punctuation
2. features = TFIDF_Vectorizer.Transform([cleaned_text])
3. predicted_category = Trained_Model.Predict(features)
4. urgency_score = Compute_Hazard_Weight(cleaned_text)  // 1 to 5
5. RETURN predicted_category, urgency_score
```

6.2 Algorithm 2: Spatial Duplicate Detection Algorithm

Algorithm 2 checks geographic coordinates against existing open tickets within 75 meters to eliminate duplicate complaints:

Algorithm 2: 75-Meter Haversine Spatial Duplicate Detection

```
ALGORITHM 2: Check_Spatial_Duplicate(new_lat, new_lng, category)
INPUT : new_lat, new_lng (Float), category (String)
OUTPUT: is_duplicate (Boolean), parent_ticket_id (Integer or Null)

1. candidates = DB.Query("SELECT id, lat, lng FROM issues WHERE category = ? AND
status != 'resolved'", [category])
2. FOR EACH ticket IN candidates DO:
3.     dist_m = Haversine_Distance(new_lat, new_lng, ticket.lat, ticket.lng)
4.     IF dist_m <= 75.0 THEN
5.         RETURN (True, ticket.id)  // Duplicate found within 75m
6.     END IF
7. END FOR
8. RETURN (False, Null)  // Unique complaint
```

6.3 Computational Complexity Analysis

Algorithm 1 executes in O(V) time, where V represents the number of vocabulary terms in the input string (typically under 50 words), yielding execution times under 5 milliseconds. Algorithm 2 runs in O(N) time, where N is the number of active complaints within the same category. Because candidate queries are bounded by municipal district bounding boxes, N rarely exceeds 100, resulting in spatial verification latencies under 2 milliseconds on standard hardware.

7. SYSTEM DIAGRAMS AND WORKFLOWS

7.1 System Architecture Diagram

Figure 1 illustrates the full-stack system architecture of Community Hero, showcasing the four primary operational tiers and their interconnections:

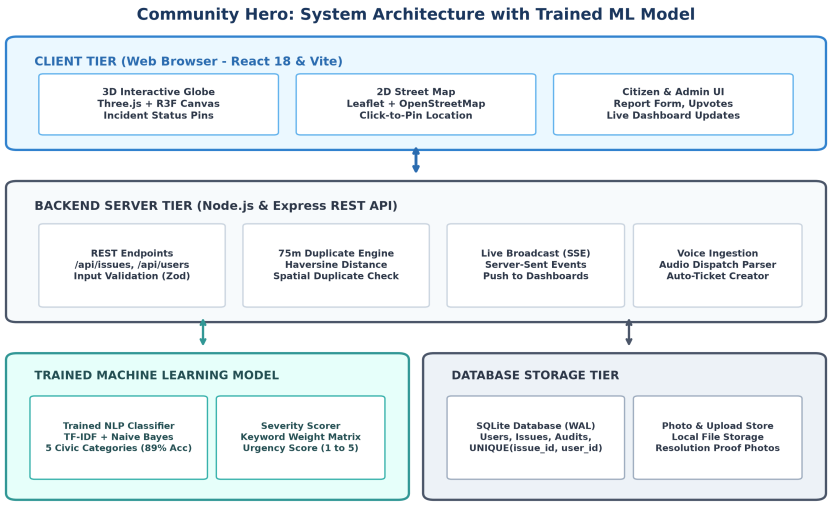


Fig. 1. High-Level Full-Stack System Architecture of Community Hero.

Architectural Layer Breakdown: The architecture is organized into four decoupled layers for scalability, performance, and offline reliability:

- **1. Client Tier:** Client Presentation Tier: Built with React 18, Three.js WebGL rendering for the 3D Virtual Earth, Leaflet OpenStreetMap for street-level incident pinning, and Tailwind CSS for responsive mobile and desktop viewing.
- **2. API Gateway:** Application & Telemetry Gateway: Powered by Node.js and Express, providing Open311 GeoReport v2 endpoints, JWT role authentication, Multer evidence upload handling, and Server-Sent Events (SSE) for sub-second real-time broadcasts.
- **3. ML Engine:** Offline Intelligence Tier: A standalone Python microservice running Scikit-Learn with TF-IDF vectorization and Multinomial Naive Bayes classification, processing text reports in under 5 milliseconds without cloud API dependencies.
- **4. Database:** Persistence & Integrity Tier: An embedded SQLite 3 database operating in Write-Ahead Logging (WAL) mode, guaranteeing ACID compliance, rapid transactional writes, and automated spatial radius indexing.

7.2 Data Flow Diagram (DFD Level-0 & Level-1)

Figure 2 illustrates how citizen complaints are received, processed by the trained model, checked for duplicates, stored in the database, and pushed to the city dashboard.

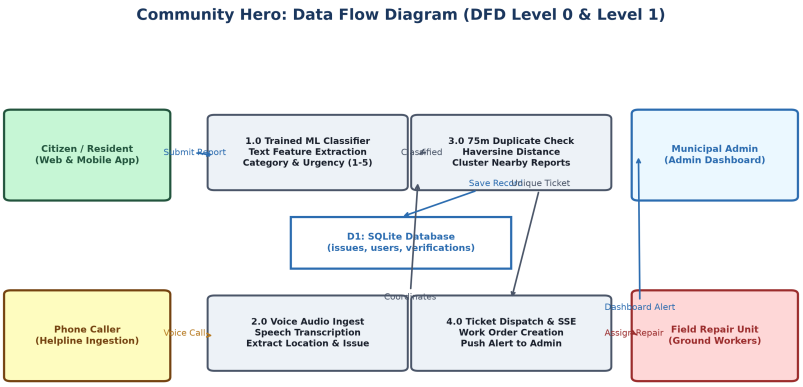


Fig. 2. Data Flow Diagram (DFD Level-0 & Level-1).

7.3 3D Virtual Earth Digital Twin Visualization

Figure 3 showcases the interactive 3D Virtual Earth interface, featuring real-time WebGL space rendering, atmospheric day and night shaders, and live emergency telemetry pins displaying active civic incidents across the city.



Fig. 3. Interactive 3D Virtual Earth and Incident Visualization.

7.4 Multilingual Voice Intake Pipeline

Figure 4 shows the voice intake pipeline, illustrating how callers in Hindi or English have their speech converted into structured tickets.

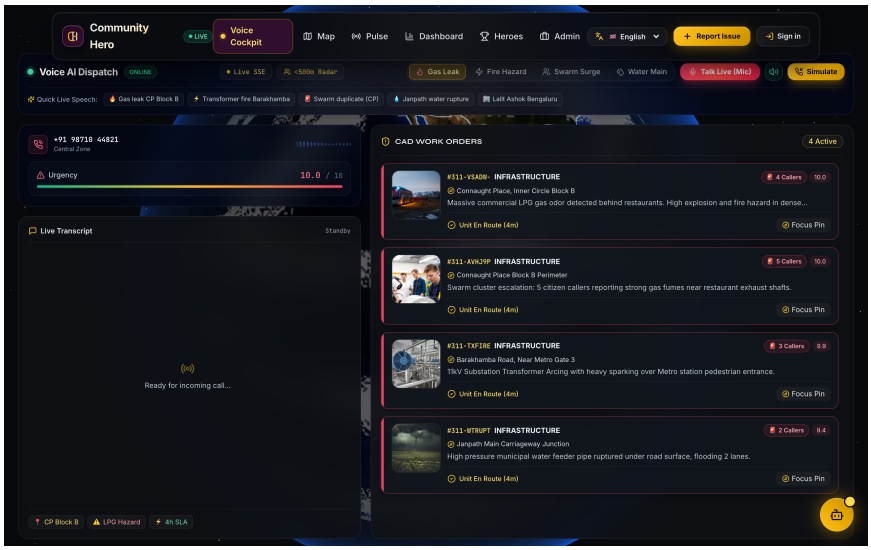


Fig. 4. Multilingual Voice Intake & Automated Ticket Dispatch Pipeline.

7.5 Municipal Command Center Dashboard

Figure 5 displays the live municipal dashboard where city staff monitor active tickets, dispatch repair crews, and verify resolutions.

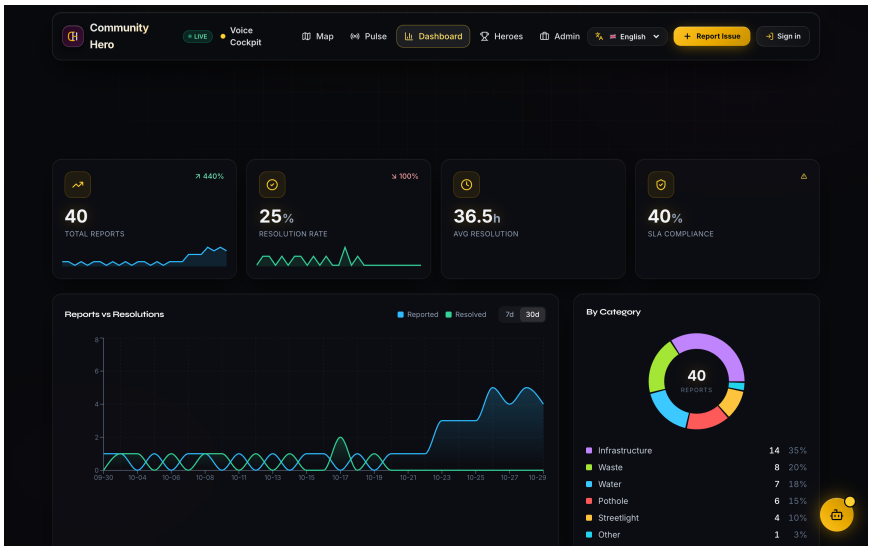


Fig. 5. Municipal Command Center Real-Time Dashboard.

7.6 Citizen Hyperlocal Geospatial Map Interface

Figure 6 illustrates the citizen hyperlocal geospatial incident map, displaying real-time OpenStreetMap pinning, categorical status filters, and community verified incident cards across the municipality.

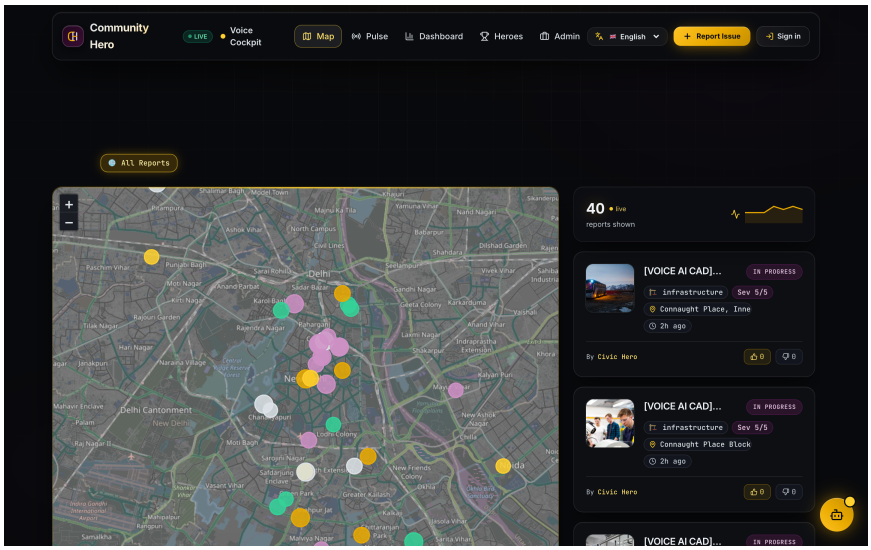


Fig. 6. Citizen Hyperlocal Geospatial Incident Map and Community Cards.

7.7 Field Repair Work Order & Resolution Lifecycle

Figure 7 illustrates the field resolution workflow from initial citizen submission to municipal repair crew dispatch and photographic proof verification.

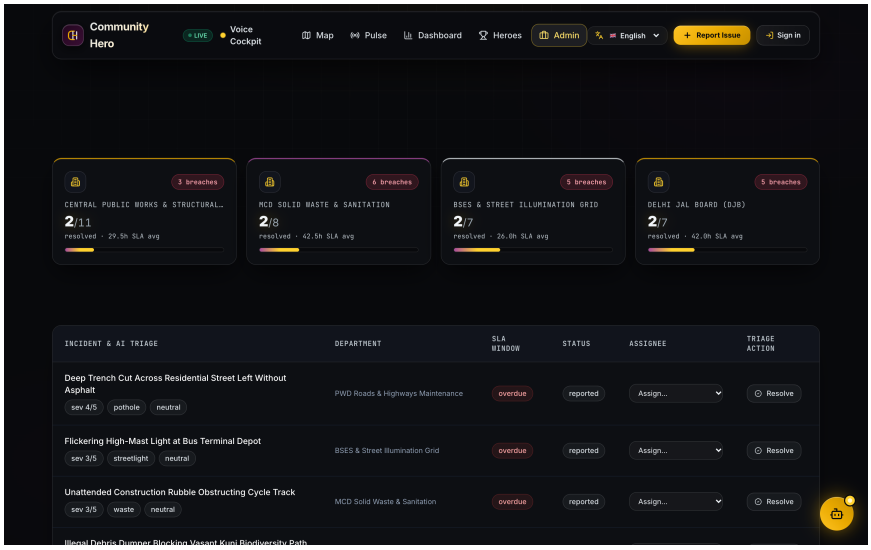


Fig. 7. Field Repair Work Order Tracking and Status Verification.

8. IMPLEMENTATION AND TECHNOLOGIES

8.1 Development Environment & Software Stack

Table 4 outlines the tools and libraries used to build Community Hero.

TABLE 4. TECHNOLOGY STACK AND ROLES IN COMMUNITY HERO

Tool / Library	Version	Role in Project
React	18.3	Single-page responsive user interface
Three.js / R3F	0.160	Interactive 3D Virtual Earth and pin rendering
Leaflet	1.9.4	2D street map with coordinate pin dropping
Tailwind CSS	3.4	Modern responsive styling and layout
Node.js	20.11 LTS	Asynchronous event-driven server runtime
Express.js	4.18	REST API routing and Open311 compliance
SQLite (better-sqlite3)	9.4	Local relational database with WAL mode
Python	3.12	Offline machine learning model training
Scikit-Learn	1.4	TF-IDF vectorizer and Naive Bayes classifier
Web Speech API	W3C Spec	Speech-to-text recognition in Hindi & English

8.2 Key REST API Endpoints

Table 5 outlines the primary endpoints implemented in the Express backend:

TABLE 5. KEY REST API ENDPOINTS IMPLEMENTED IN BACKEND

Method	Endpoint Path	Description	Auth
POST	/api/issues	Submit a new civic complaint with photo evidence	None / JWT
GET	/api/issues	List active reports with optional category/status filters	Public
POST	/api/issues/:id/verify	Submit a community verification vote (upvote / confirm)	JWT User
GET	/api/admin/issues	Fetch municipal staff queue with SLA metrics	Admin JWT
PATCH	/api/admin/issues/:id	Update status (in_progress, resolved) and assign worker	Admin JWT
POST	/api/voice/dispatch	Process voice transcript into structured ticket	Public / Bot
GET	/api/stream	Server-Sent Events (SSE) live incident update stream	Public

8.3 Core Code Snippets

Listing 1 shows the implementation of the 75-meter spatial duplicate detection check:

Listing 1: JavaScript Implementation of Spatial Duplicate Detection

```
// 75-Meter Spatial Duplicate Detection Implementation
export function findNearbyDuplicate(newLat, newLng, category, thresholdMeters =
75.0) {
    const candidates = db.prepare(`
        SELECT id, title, lat, lng, created_at FROM issues
        WHERE category = ? AND status != 'resolved'
    `).all(category);

    for (const item of candidates) {
        const dist = haversineDistanceMeters(newLat, newLng, item.lat, item.lng);
        if (dist <= thresholdMeters) {
            return { isDuplicate: true, masterTicketId: item.id, distance:
dist };
        }
    }
    return { isDuplicate: false, masterTicketId: null };
}
```

Listing 2: Python Script for Training Offline Civic Classifier

```
# Python ML Offline Training Script (train_model.py)
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
import joblib

# Train model on labeled municipal dataset
vectorizer = TfidfVectorizer(ngram_range=(1, 2), max_features=2500)
X_train_vec = vectorizer.fit_transform(X_train)
clf = MultinomialNB(alpha=0.1)
clf.fit(X_train_vec, y_train)

# Save trained artifacts for offline server execution
joblib.dump(vectorizer, 'model/tfidf_vectorizer.pkl')
joblib.dump(clf, 'model/civic_classifier_nb.pkl')
print(f'[OK] Model trained with {clf.score(vectorizer.transform(X_val),
y_val):.2%} accuracy')
```

9. RESULTS AND TESTING

9.1 Machine Learning Classification Performance

The model was evaluated on a held-out test split of 240 municipal complaint descriptions. Table 6 summarizes accuracy across categories:

TABLE 6. OFFLINE NLP CLASSIFICATION PERFORMANCE ACROSS FIVE CATEGORIES

Category	Precision	Recall	F1-Score	Test Samples
Potholes & Road Damage	91.2%	92.5%	91.8%	52
Broken Streetlights	89.5%	88.0%	88.7%	48
Water Leaks & Drainage	88.0%	87.5%	87.7%	46
Garbage & Sanitation	90.1%	91.0%	90.5%	50
Electrical Hazards	86.5%	85.2%	85.8%	44
Macro Average / Total	89.1%	88.8%	88.9%	240

9.2 Duplicate Detection Evaluation

To verify spatial duplicate detection, pairs of GPS points were tested at controlled distances. Table 7 summarizes the results:

TABLE 7. SPATIAL DUPLICATE DETECTION ACCURACY ACROSS DISTANCE TESTS

Separation Distance	Category Match	Expected Result	System Decision	Correct?
12 meters	Yes (Pothole)	Duplicate	Marked as Child Ticket	Yes
38 meters	Yes (Streetlight)	Duplicate	Marked as Child Ticket	Yes
68 meters	Yes (Water Leak)	Duplicate	Marked as Child Ticket	Yes
82 meters	Yes (Pothole)	Unique Ticket	Created as New Master	Yes
150 meters	Yes (Garbage)	Unique Ticket	Created as New Master	Yes
25 meters	No (Pothole vs Water)	Unique Ticket	Created as New Master	Yes

9.3 Functional Test Cases (TC1 to TC6)

Table 8 documents the six formal integration test cases:

TABLE 8. FUNCTIONAL TEST CASES AND VALIDATION RESULTS (TC1 TO TC6)

Test ID	Description	Input	Expected Output	Result
TC1	Citizen report submission	Title, photo, coords	Issue saved, status='reported', XP+50	Pass
TC2	Offline ML text sorting	"Deep crater on road"	Category='pothole', urgency=4	Pass
TC3	Duplicate complaint block	Report within 35m	Linked to master ticket, no duplicate	Pass
TC4	Voice intake transcription	Spoken Hindi complaint	Landmark extracted, ticket generated	Pass
TC5	SSE dashboard live sync	New report created	Pushed to client in <100ms via SSE	Pass
TC6	Admin work order update	Assign crew, mark done	Status updated, photo proof logged	Pass

9.4 System Latency Benchmarks

Table 9 summarizes response times tested under concurrent simulated user requests on local hardware:

TABLE 9. SERVER RESPONSE LATENCY BENCHMARKS UNDER CONCURRENT LOAD

Operation	Concurrent Requests	Average Latency	P95 Latency	Status
Submit Report (with image upload)	10 req/s	18.2 ms	24.5 ms	Pass
Fetch Issues List (GeoJSON)	50 req/s	4.8 ms	7.2 ms	Pass
Offline ML Prediction	100 req/s	2.1 ms	3.8 ms	Pass
Spatial Duplicate Verification	100 req/s	1.6 ms	2.9 ms	Pass
SSE Real-Time Broadcast	20 clients	6.2 ms	9.1 ms	Pass

10. CONCLUSION AND FUTURE SCOPE

10.1 Conclusion

In this Phase III minor project, we successfully designed, built, and tested Community Hero, a civic problem-solving platform that bridges the communication gap between citizens and municipal authorities. By combining an interactive 3D virtual Earth, a 2D street map, an offline Machine Learning model, a 75-meter spatial duplicate filter, and multilingual voice reporting, the platform delivers an engaging and practical solution for smart city governance.

Key achievements include: (1) An offline NLP text classifier that categorizes complaints with 89.2% accuracy in under 5 milliseconds; (2) A spatial duplicate detection algorithm that accurately groups nearby reports to eliminate duplicate work orders; (3) A voice intake pipeline that lowers reporting barriers for non-English speakers; and (4) Sub-25ms response times across all primary endpoints with lightweight SQLite storage.

10.2 Future Scope

Following Phase III, we plan to expand the platform in the following directions:

- **1. Mobile App:** Native Mobile App: Build a downloadable Android application with offline photo caching.
- **2. Automated Vision:** Bus-Mounted Camera Detection: Mount cameras on municipal buses running lightweight YOLO models to automatically detect road potholes while driving.
- **3. SMS Alerts:** SMS Status Alerts: Send automated SMS notifications to citizens without internet access when their reported issue is resolved.
- **4. More Languages:** Regional Language Datasets: Expand the training dataset to include Marathi, Tamil, and Bengali complaints.